Итог

Статус: Не готово к сдаче.

По содержанию реализация хорошо соответствует ТЗ: выбран датасет для обнаружения объектов, подготовлены данные в формате YOLO, предусмотрены обучение YOLO, подбор гиперпараметров, валидация, тестирование, визуализация, анализ ошибок, сравнение с Faster R-CNN и демонстрация IoU. Однако файл practical_work_2_yolo.ipynb сохранён полностью без результатов выполнения: у всех 23 кодовых ячеек execution_count равен None, а outputs отсутствуют. Поэтому нельзя подтвердить фактическое обучение, значения метрик, построение графиков и получение предсказаний. Кроме того, реализация не использует Roboflow API и Google Drive, хотя они прямо указаны в ТЗ. После полного выполнения ноутбука в Colab и сохранения выходов работа, вероятно, будет готова к сдаче с небольшими замечаниями.

Матрица соответствия

№	Источник	Требование	Статус	Доказательство	Замечание
1	ТЗ	Использовать YOLOv26 для обнаружения объектов.	частично	Кодовые ячейки 21-34: используются yolo26n.pt и yolo26s.pt через Ultralytics YOLO.	Код присутствует, но обучение и результаты в файле не сохранены.
2	ТЗ	Использовать Python 3 и Ultralytics YOLO.	выполнено	Ячейки 3-4: установка ultralytics, импорт YOLO и проверка окружения.	Соответствие подтверждается кодом.
3	ТЗ	Использовать PyTorch или TensorFlow.	выполнено	Ячейки 3-4 и 35-43: импорт torch, torchvision и реализация Faster R-CNN.	Используется PyTorch.
4	ТЗ	Использовать NumPy, Matplotlib, Seaborn и OpenCV.	выполнено	Ячейки импорта и последующие функции анализа и визуализации.	Все перечисленные библиотеки используются в коде.
5	ТЗ	Выбрать или создать размеченный датасет.	выполнено	Ячейка 7: выбран датасет BCCD с классами RBC, WBC и Platelets.	Датасет подходит для обнаружения объектов.
6	ТЗ	Использовать готовый датасет из Roboflow Universe либо собственную разметку.	частично	Ячейки 7 и 9: используется BCCD, загружаемый из GitHub.	Размеченный набор выбран, но источник Roboflow Universe не подтверждён.
7	ТЗ / инструменты	Использовать Roboflow API для загрузки и разметки данных.	не выполнено	В ноутбуке отсутствуют импорт roboflow, API-вызовы и ключ проекта.	Данные клонируются из GitHub.
8	ТЗ / этап 1	Загрузить данные на Google Drive для сохранения между перезапусками Colab.	не выполнено	Используются пути внутри /content; монтирование Drive отсутствует.	Данные и артефакты исчезнут после сброса среды Colab.
9	ТЗ	Загрузить и предобработать данные.	выполнено	Ячейки 9-16: загрузка, чтение VOC XML, очистка, преобразование разметки и статистика.	Предобработка подробно реализована.
10	ТЗ	Проверить соответствие изображений и аннотаций.	выполнено	Ячейки 9, 11 и 14: проверка пар, размеров, классов и допустимости bounding box.	Есть автоматические проверки и assert.
11	ТЗ	Проверить корректность bounding box.	выполнено	Функции чтения VOC и преобразования координат ограничивают рамки границами изображения и исключают вырожденные рамки.	Обработка пограничных случаев предусмотрена.
12	ТЗ	Визуализировать изображения с bounding box.	частично	Ячейка 18: функция визуализации истинных рамок.	Код есть, но сохранённого изображения или вывода нет.
13	ТЗ	Привести изображения к одинаковому разрешению, например 640x640.	выполнено	В конфигурации обучения используется imgsz=640; преобразование выполняется средствами Ultralytics и torchvision.	Требование реализовано через пайплайн модели.
14	ТЗ	Разделить данные на train, validation и test.	выполнено	Ячейки 9-16: детерминированное разбиение 70/15/15 и проверка отсутствия пересечений.	Разделение соответствует рекомендуемой пропорции.
15	ТЗ	Подготовить структуру YOLO и YAML-конфигурацию.	выполнено	Ячейки 13-16: создаются images/labels для трёх частей и data.yaml.	Формат соответствует Ultralytics YOLO.

№	Источник	Требование	Статус	Доказательство	Замечание
16	ТЗ	Обучить базовую модель YOLO.	частично	Ячейки 22-24: конфигурация и вызов model.train для yolo26n.pt.	Нет журналов обучения и обученного checkpoint в приложенном файле.
17	ТЗ	Оценить Precision, Recall и mAP на validation.	частично	Ячейки 25-30: извлечение метрик из результатов Ultralytics.	Алгоритм предусмотрен, фактические значения отсутствуют.
18	ТЗ	Оценить модель на test.	частично	Ячейки 32-34: model.val с split=test для выбранной модели.	Нет сохранённого результата выполнения.
19	ТЗ	Построить графики loss и mAP.	частично	Ячейки 25-30: предусмотрено чтение results.csv и построение кривых.	Графики в outputs отсутствуют.
20	ТЗ	Подобрать оптимальные гиперпараметры.	выполнено	Ячейки 21 и 31: три эксперимента с yolo26n/yolo26s, batch, lr0, optimizer и augmentation.	Подбор реализован программно.
21	ТЗ	Выбрать лучшую конфигурацию по validation.	выполнено	Ячейки 29-31: таблица экспериментов и выбор по mAP50-95 validation.	Test не используется для выбора модели.
22	ТЗ	Протестировать модель на новых изображениях.	частично	Ячейки 44-47: предсказания на четырёх изображениях test-набора.	Используется отложенная test-выборка, но фактические изображения с предсказаниями не сохранены.
23	ТЗ	Сохранить bounding box и confidence scores в файл.	выполнено	Ячейка 46: формируется predictions.csv с координатами, классом и confidence.	Код сохранения присутствует.
24	ТЗ	Проанализировать False Positives, False Negatives и сложные случаи.	частично	Ячейки 48-50: подсчёт TP/FP/FN, ошибок локализации и визуализация худших случаев.	Код есть, но фактический анализ без выполнения не подтверждён.
25	Методичка / задание после работы	Сравнить разные размеры YOLO.	частично	Ячейки 21 и 31: сравниваются nano и small.	Medium не рассматривается, а результаты не сохранены.
26	Методичка / задание после работы	Сравнить YOLO с Faster R-CNN или другой моделью.	частично	Ячейки 35-43: реализованы обучение и оценка Faster R-CNN.	Сравнение предусмотрено, но результатов выполнения нет.
27	Методичка / опционально	Реализовать IoU вручную и сравнить с библиотечной реализацией.	выполнено	Ячейки 51-52: собственная функция IoU, сравнение с torchvision.ops.box_iou и визуализация.	Статические чистые проверки функции прошли.
28	ТЗ / формат	Представить работу в Jupyter Notebook с пояснениями Markdown.	выполнено	practical_work_2_yolo.ipynb: 54 ячейки, из них 31 Markdown и 23 кодовые.	Структура и пояснения присутствуют.
29	ТЗ / отчёт	Описать этапы, результаты, визуализации и выводы.	частично	В ноутбуке есть последовательные разделы и программно формируемый итоговый вывод.	Фактические результаты и выводы не сохранены из-за отсутствия outputs.
30	Качество	Обеспечить воспроизводимость.	выполнено	Заданы seed для random, NumPy, PyTorch; разбиение выполняется детерминированно.	Базовая воспроизводимость предусмотрена.
31	Качество	Не хранить секреты и пользовательские данные.	выполнено	Статический поиск не выявил токенов, паролей, ключей API или персональных данных.	Секретов в ноутбуке нет.
32	Качество	Предоставить доказательства фактического выполнения.	не выполнено	У всех 23 кодовых ячеек execution_count=None; общее число outputs равно 0.	Главный блокер для приёмки.
33	Спецификация / тикеты	Проверить дополнительные документы проекта.	не проверено	По указанию пользователя аудит ограничен двумя приложенными notebook-файлами.	Отсутствие дополнительных документов не считается ошибкой.

Найденные проблемы

AUD-001 - Critical

Источник: Качество / доказательства выполнения

Описание: Ноутбук сохранён без единого результата выполнения. Нельзя подтвердить обучение YOLO и Faster R-CNN, метрики, графики, предсказания, CSV и итоговый вывод.

Как воспроизвести или проверить: Открыть practical_work_2_yolo.ipynb и проверить execution_count и outputs всех кодовых ячеек.

Ожидаемый и фактический результат: Ожидается выполненный notebook с логами, таблицами, графиками и изображениями. Фактически у 23 из 23 кодовых ячеек execution_count=None и outputs отсутствуют.

Рекомендуемое исправление: Выполнить Runtime -> Run all в Google Colab с GPU, дождаться завершения всех этапов и сохранить notebook вместе с outputs.

AUD-002 - Major

Источник: ТЗ / используемые инструменты

Описание: Roboflow API не используется. Датасет BCCD загружается клонированием GitHub-репозитория.

Как воспроизвести или проверить: Найти в ноутбуке импорт roboflow, Roboflow API key или вызовы workspace/project/version/download.

Ожидаемый и фактический результат: ТЗ перечисляет Roboflow API для загрузки и разметки. В реализации соответствующего кода нет.

Рекомендуемое исправление: Либо загрузить выбранную версию датасета через Roboflow API, либо заранее согласовать с преподавателем использование эквивалентного источника GitHub.

AUD-003 - Major

Источник: ТЗ / этап подготовки данных

Описание: Google Drive не подключается; данные, веса и артефакты хранятся во временной файловой системе /content.

Как воспроизвести или проверить: Проверить наличие drive.mount и путей /content/drive.

Ожидаемый и фактический результат: По ТЗ данные должны сохраняться между перезапусками Colab. Фактически содержимое /content будет удалено после сброса runtime.

Рекомендуемое исправление: Добавить сохранение датасета и итоговых артефактов на Google Drive либо подготовить явное скачивание notebook, весов, CSV и графиков до завершения сессии.

AUD-004 - Major

Источник: Качество сравнения моделей

Описание: Время обучения YOLO и Faster R-CNN измеряется неодинаково. Вызов model.train включает внутреннюю инфраструктуру Ultralytics и валидацию, а собственный цикл Faster R-CNN измеряет другой объём работы.

Как воспроизвести или проверить: Сравнить участки измерения времени в ячейках обучения YOLO и Faster R-CNN.

Ожидаемый и фактический результат: Для честного сравнения одинаковая метрика времени должна охватывать сопоставимые этапы. Фактически интервалы неэквивалентны.

Рекомендуемое исправление: Либо измерять одинаковый участок train+validation для обеих моделей, либо явно указать в выводах, что время приведено ориентировочно и напрямую несопоставимо.

AUD-005 - Major

Источник: Качество выбора модели

Описание: YOLO оценивается по best.pt, а Faster R-CNN после обучения используется в состоянии последней эпохи без выбора лучшего checkpoint по validation.

Как воспроизвести или проверить: Проверить выбор checkpoint для YOLO и состояние frcnn_model перед тестированием.

Ожидаемый и фактический результат: Ожидается сопоставимый принцип выбора моделей. Фактически для YOLO берётся лучшая эпоха, а для Faster R-CNN последняя.

Рекомендуемое исправление: Сохранять лучший Faster R-CNN checkpoint по validation mAP50-95 и использовать его на test.

AUD-006 - Minor

Источник: Воспроизводимость

Описание: Версия Ultralytics не зафиксирована, а BCCD клонируется из текущего HEAD репозитория.

Как воспроизвести или проверить: Проверить команду установки и команду git clone.

Ожидаемый и фактический результат: Ожидается повторяемое окружение и неизменяемая версия данных. Фактически результат может измениться после обновления пакета или репозитория.

Рекомендуемое исправление: Зафиксировать версию ultralytics и commit/tag датасета.

AUD-007 - Minor

Источник: Анализ ошибок

Описание: На визуализации сложных примеров показываются все истинные рамки одним цветом и все предсказания другим, но конкретная причина ошибки - FP, FN или плохая локализация - визуально не выделяется.

Как воспроизвести или проверить: Выполнить ячейки анализа ошибок и открыть итоговый рисунок.

Ожидаемый и фактический результат: Ожидается наглядное указание проблемной рамки или типа ошибки. Фактически зрителю нужно самостоятельно сопоставлять рамки.

Рекомендуемое исправление: Подписать тип ошибки и выделять несопоставленные или плохо локализованные рамки отдельными цветами.

Непроверяемые пункты

- Фактическая загрузка и преобразование всех 364 изображений BCCD - нужен выполненный вывод ячеек подготовки данных.
- Успешное завершение трёх экспериментов YOLO - нужны журналы model.train и каталоги runs.
- Доступность checkpoint yolo26n.pt и yolo26s.pt в фактической версии Ultralytics среды Colab - нужен вывод запуска ячеек загрузки моделей.
- Реальные Precision, Recall, mAP50 и mAP50-95 на validation и test - нужны сохранённые таблицы metrics.
- Метрики по каждому классу - нужен вывод таблицы per-class metrics.
- Обучение Faster R-CNN без ошибки памяти GPU - нужен журнал обучения в Colab.
- Сравнение скорости, числа параметров и размера checkpoint - нужна итоговая выполненная таблица сравнения.

- Фактические примеры FP, FN и плохой локализации - нужен сохранённый рисунок анализа ошибок.
- Создание predictions.csv - нужен сам файл или вывод его первых строк.
- Успешный сценарий Runtime -> Run all - нужен полностью выполненный notebook без tracebacks.
- Тесты или документы текущей папки проекта не проверялись, поскольку пользователь явно ограничил источники двумя приложенными notebook-файлами.

Лишняя реализация

- Защитная обработка вырожденных и выходящих за границы изображения bounding box.
- Расчёт COCO-метрик по каждому классу.
- Сравнение времени инференса, числа параметров и размера checkpoint.
- Автоматический выбор лучшего эксперимента только по validation.
- Дополнительное сравнение Precision и Recall при общем confidence threshold.
- Подробный подсчёт TP, FP, FN и ошибок локализации.
- Численная и визуальная демонстрация IoU.
- Программно формируемый итоговый вывод по результатам экспериментов.

Faster R-CNN и IoU не являются лишними в отрицательном смысле: они прямо предложены в дополнительных заданиях ТЗ и повышают полноту работы.

Результаты проверок

Проверка структуры и контрольных сумм файлов: ТЗ: размер 11 783 байта, SHA256 EDFAA85DD2586410B283372D70E8856116BD53D1CFEC89DEA086C20CA4A963C2, nbformat 4.0, 2 ячейки. Реализация: размер 103 921 байт, SHA256 14B45F84758BFB34BC3B01F2F8C3B54F1507D880EB9FCD31569A27D016153D88, nbformat 4.5, 54 ячейки.

nbformat.validate: Успешно для обоих notebook-файлов.

Подсчёт Markdown, code, execution_count и outputs: Реализация содержит 31 Markdown-ячейку и 23 кодовые ячейки; выполненных кодовых ячеек 0; outputs 0.

AST-разбор Python-кода: Все 23 кодовые ячейки успешно разобраны после исключения Jupyter magics.

Поиск секретов: Токены, пароли, API-ключи и иные очевидные секреты не обнаружены.

Чистые проверки координат и IoU: Пройдены: преобразование координат и функция IoU дают ожидаемые значения.

Синтетическая проверка precision_recall_at_threshold: Пройдена; получен ожидаемый результат precision=0.5, recall=1.0, TP=1, FP=1, FN=0.

Полное обучение и сетевые операции: Не выполнялись: это ресурсоёмкие операции, а notebook не содержит сохранённых результатов.

Отдельные промежуточные команды ревьюера сначала завершались ошибкой из-за обработки Jupyter magics и рабочего каталога; после корректировки соответствующие проверки прошли. Эти ошибки не относятся к реализации notebook.

Подготовка к защите

Сценарий демонстрации работы по шагам

1. Открыть notebook в Google Colab и включить GPU.
2. Показать версии Python, PyTorch, CUDA и Ultralytics.
3. Запустить загрузку BCCD и показать структуру исходных данных.
4. Показать разбиение 70/15/15 и доказательство отсутствия пересечений между train, validation и test.
5. Показать статистику классов и результаты проверки изображений и аннотаций.
6. Показать несколько исходных изображений с истинными bounding box.
7. Показать обучение базовой YOLOv26 nano и сохранённые кривые loss и mAP.
8. Показать таблицу трёх экспериментов и объяснить выбор лучшей конфигурации по validation mAP50-95.
9. Запустить оценку выбранной YOLO на test и показать Precision, Recall, mAP50 и mAP50-95.
10. Показать обучение и оценку Faster R-CNN, затем итоговую таблицу сравнения.
11. Продемонстрировать ручной расчёт IoU и совпадение с torchvision.ops.box_iou.
12. Показать четыре тестовых изображения с предсказанными рамками и confidence.
13. Открыть predictions.csv и показать несколько строк.
14. Показать разбор FP, FN и ошибок локализации и завершить итоговым выводом.

Обязательные экраны, команды и тестовые данные

- Экран с GPU и версиями библиотек.
- Количество изображений в train, validation и test.
- Распределение объектов по классам.
- Результат проверки разметки и отсутствия утечки данных.
- Примеры исходной разметки.
- Кривые box loss, classification loss и mAP.
- Confusion matrix, PR curve и F1 curve Ultralytics.
- Таблица гиперпараметров и validation-метрик трёх YOLO-экспериментов.
- Test-метрики выбранной модели и метрики по классам.
- Таблица YOLO против Faster R-CNN.
- Численный и графический пример IoU.
- Четыре тестовых изображения с рамками и confidence.

- Визуализация ошибок и файл predictions.csv.

Доказательства выполнения ключевых требований

- Полностью выполненный notebook с сохранёнными outputs.
- Логи обучения каждой конфигурации.
- Таблицы validation и test метрик.
- Checkpoint лучшей модели.
- Графики обучения и оценки.
- Примеры предсказаний на test-изображениях.
- predictions.csv с координатами, классом и confidence.
- Итоговый вывод, основанный на реально полученных числах.

10 вероятных вопросов преподавателя и краткие ответы

1. Почему выбран BCCD? Это небольшой размеченный датасет обнаружения объектов с тремя классами крови - RBC, WBC и Platelets. Он содержит несколько объектов на изображении и подходит для сравнения детекторов.

2. Что представляет собой формат YOLO? Для каждого изображения используется txt-файл, где каждая строка содержит class_id и нормализованные координаты центра, ширину и высоту рамки.

3. Как выполнено преобразование VOC в YOLO? Из XML читаются xmin, ymin, xmax, ymax, затем вычисляются центр, ширина и высота и нормализуются на размер изображения.

4. Как предотвращена утечка данных? Разбиение выполняется по уникальным stem изображений с фиксированным seed, после чего assert проверяет отсутствие пересечений между частями.

5. Чем mAP50 отличается от mAP50-95? mAP50 рассчитывается при IoU 0.5, а mAP50-95 усредняется по порогам IoU от 0.5 до 0.95 и строже оценивает качество локализации.

6. Почему лучшая модель выбирается по validation? Validation используется для выбора конфигурации, чтобы test оставался независимой оценкой итоговой модели.

7. Какие гиперпараметры сравниваются? Размер модели nano или small, batch size, начальный learning rate, optimizer и набор аугментаций.

8. Чем YOLO отличается от Faster R-CNN? YOLO является одностадийным детектором и обычно быстрее, а Faster R-CNN - двухстадийный детектор, сначала предлагающий регионы, а затем классифицирующий их.

9. Как считаются TP, FP и FN? Предсказание сопоставляется с ещё не использованной истинной рамкой того же класса; при IoU не ниже 0.5 это TP, несопоставленное предсказание - FP, несопоставленная истинная рамка - FN.

10. Какие фактические метрики получены? В приложенном состоянии notebook фактические метрики отсутствуют, поскольку ячейки не выполнены. До защиты notebook необходимо запустить и отвечать по сохранённым результатам.

Что обязательно исправить или подготовить до сдачи

- Полностью выполнить notebook в Colab с GPU и сохранить все outputs.
- Проверить, что все три YOLO-эксперимента и Faster R-CNN заверются без ошибок.

- Сохранить реальные validation и test метрики, графики и примеры предсказаний.
- Подготовить checkpoint лучшей модели и predictions.csv.
- Согласовать с преподавателем допустимость загрузки BCCD из GitHub вместо Roboflow API.
- Согласовать допустимость работы без Google Drive или добавить постоянное хранение артефактов.

Решение по приёмке

Обязательно исправить до сдачи

- Запустить notebook полностью и сохранить результаты выполнения.
- Предъявить реальные validation и test метрики YOLO.
- Подтвердить успешное обучение и оценку Faster R-CNN либо убрать неподтверждённые сравнительные выводы.
- Сохранить обязательные графики, предсказания и анализ ошибок.

Желательно исправить

- Сделать измерение времени YOLO и Faster R-CNN сопоставимым.
- Выбирать лучший Faster R-CNN checkpoint по validation.
- Зафиксировать версии Ultralytics и датасета.
- На визуализации ошибок явно выделять FP, FN и ошибки локализации.

Можно объяснить как ограничение работы

- Использование BCCD из GitHub вместо Roboflow при наличии согласования преподавателя.
- Хранение в /content вместо Google Drive, если все результаты скачиваются до завершения сессии.
- Сравнение только nano и small без medium.
- Использование изображений отложенной test-выборки как новых изображений.
- Ограниченный перебор гиперпараметров из-за вычислительных ресурсов Colab.

Требуется уточнение у преподавателя

- Является ли Roboflow API строго обязательным или допустим эквивалентный размеченный датасет из GitHub?
- Является ли Google Drive обязательным элементом или только рекомендацией для Colab?
- Достаточно ли сравнения YOLO nano и small для дополнительного задания?
- Считаются ли изображения из test-набора новыми изображениями для демонстрации?
- Нужно ли прикладывать checkpoint и predictions.csv отдельно от notebook?
- В тексте ТЗ комментарии к metrics.box.map и metrics.box.map50 перепутаны: в Ultralytics map означает mAP50-95, а map50 - mAP50. Реализация использует корректную семантику; стоит уточнить, как преподаватель ожидает видеть подписи.